

# Reviewer Pack — Symmetric Power Rank Gap in CNF Incidence Matrices

Entry 2e234361503d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 22:53:49 UTC

## Symmetric Power Rank Gap in CNF Incidence Matrices

**Entry ID:** 2e234361503d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 22:53:49 UTC

### 1. Conjecture

**Field A** (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Boolean Circuit Complexity

#### Statement:

For a random 3-CNF formula  $\Phi$  on  $n$  variables, the rank of the symmetric square of its incidence matrix  $M$  satisfies  $\text{rank}(M^{\otimes 2}) \geq \Omega(n^{\{1.5\}})$ , whereas for a random 2-CNF formula  $\Psi$  on  $n$  variables,  $\text{rank}(M^{\otimes 2}) = O(n^{\{1.2\}})$  with probability  $\geq 0.95$  over 30 seeds.

#### Rationale (proposer’s reasoning):

Symmetric powers of incidence matrices encode combinatorial structure that may correlate with circuit complexity. The gap in rank growth between 3-CNF and 2-CNF could reflect inherent differences in their algebraic representations under Schur-Weyl duality.

**Taxonomy category:** GCT\_DET\_PERM (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 7c6086fd412f18d6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | SAFE      | 0.95       | UNCERTAIN        | SAFE             |
| NATURAL_PROOFS | SAFE      | 0.00       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | UNCERTAIN | 0.95       | HITS             | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.95       | SAFE             | SAFE             |

| Barrier | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|---------|---------|------------|------------------|------------------|
|---------|---------|------------|------------------|------------------|

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda r: abs(A[r][i]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def rank(A):
    A = gaussian_elimination(A)
    r = sum(1 for row in A if any(row))
    return r

def incidence_matrix(cnf):
    n = max(abs(var) for clause in cnf for var in clause)
    M = [[0] * (n + 1) for _ in range(len(cnf))]
    for i, clause in enumerate(cnf):
        for var in clause:
            M[i][abs(var)] += 1 if var > 0 else -1
    return M

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    cnf3 = [[random.randint(-n, n) for _ in range(3)] for _ in range(n)]
    cnf2 = [[random.randint(-n, n) for _ in range(2)] for _ in range(n)]

    M3 = incidence_matrix(cnf3)
    M2 = incidence_matrix(cnf2)

    rank_M3 = rank(M3)
    rank_M2 = rank(M2)

    metric_name = "rank(M^2)"
```

```

metric_value = rank_M3
instances_tested = 1
conjecture_holds = rank_M3 >= n**1.5 and rank_M2 <= n**1.2
counterexample = "" if conjecture_holds else f"3-CNF: {rank_M3}, 2-CNF: {rank_M2}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in enumerate(results) if not r["conjecture_holds"])
        counterexample = results[first_failing_seed]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a13cfc5c.py", line 75, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a13cfc5c.py", line 52, in run_trial
    rank_M3 = rank(M3)
             ^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a13cfc5c.py", line 30, in rank
    A = gaussian_elimination(A)
      ^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a13cfc5c.py", line 24, in gaussian_elimination
    factor = A[j][i] / A[i][i]

```

~~~~~^~~~~~  
ZeroDivisionError: float division by zero

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed with division by zero error, preventing data collection. Cannot assess conjecture validity without successful runs. | next: Debug Gaussian elimination implementation to handle singular matrices, then re-run with 30 seeds

## 11. Audit log (LLM calls)

**Total LLM calls:** 7

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 42217        |
| 2 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 24284        |
| 3 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 20754        |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 15827        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13013        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8933         |
| 7 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 18051        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 143078 ms total latency. Provider mix: {'ollama\_remote': 7}

(full prompt+response transcripts available in *research/audit/2e234361503d.jsonl*)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/2e234361503d.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** *research/pvsnp\_sandbox/test\_\*.py* (per-cycle)

**Replay tarball:** *research/replay/2e234361503d.tar.gz* (if generated)